EXPERIMENT 13

Case Study on Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Name: *Kanishka S* | ***Register Number:*** *240701227*

Title

Case Study on Limitations of Low-Code Platforms for a Complex Enterprise Use Case – Enterprise Campus Sustainability Management System.

Aim

To study the limitations of low-code platforms when they are applied to complex enterprise-level applications, using the Smart Campus Sustainability & Carbon Tracker application built in Zoho Creator as the starting point, and to understand when pro-code extensions or a hybrid architecture become necessary.

Objectives

- To describe how an enterprise-scale Campus Sustainability Management System is structured, from users to analytics.
- To map the standard low-code capabilities (forms, reports, workflows, data model, roles) to the operations of the system.
- To investigate the engineering ceilings, bottlenecks and scaling limits of low-code at enterprise scale through eight core limitations.
- To present a three-tier hybrid architecture (low-code application, pro-code extensions, enterprise systems) as the resolution.
- To judge when the choice between low-code, pro-code and hybrid development should be made.

Software / Tools Required

- Zoho Creator (web-based low-code application development platform)
- Smart Campus Sustainability & Carbon Tracker application (Energy_Consumption, Water_Consumption, Waste_Management, Sustainability_Complaints and other forms) created in Zoho Creator
- Web browser with an active internet connection

Introduction

Low-code platforms allow applications to be developed mainly through visual tools instead of large amounts of traditional program code. Applications are assembled from predefined components such as forms, reports and dashboards, and behaviour is defined through workflows, rules and configuration, with a small amount of scripting for specific needs. This reduces development time and effort and lets a wider range of people take part in building business applications.

Low-code platforms suit standard business applications. As the requirements of an enterprise grow in users, data volume, process complexity, integrations, security and customisation, the built-in capabilities of a platform may no longer be sufficient on their own. This case study examines such limitations by scaling up the Smart Campus Sustainability & Carbon Tracker from a single campus to an enterprise-wide

network of campuses.

Case Study: Enterprise Campus Sustainability Management System

The Smart Campus Sustainability & Carbon Tracker records electricity, water and waste data for campus buildings, estimates carbon emissions using an emission factor, tracks green activities and student participation, handles sustainability complaints, and presents KPI tiles and charts on the Sustainability Dashboard. Access is controlled through the role hierarchy Sustainability Officer → Facility Coordinator → Data Entry Staff and a Read Only data-sharing rule. Suppose an education group must now run sustainability monitoring across many campuses and thousands of buildings throughout the year. The system would then serve the following users and functions:

Category	Items
Users	Students and staff, facility staff, data entry staff, sustainability officers, campus heads and administrators
Data and operations	Building records, hourly or daily energy, water and waste readings, emission factors, green activity and participation records, complaint records and resolution history
Supporting functions	Alerts for abnormal consumption, complaint notifications, status updates and AI-assisted answers
Enterprise integration and insight	Smart-meter / IoT data, CRM integration, utility billing integration, analytics and reporting

Figure 13.1 shows the architecture of this system on a low-code platform, arranged as six groups connected from left to right.

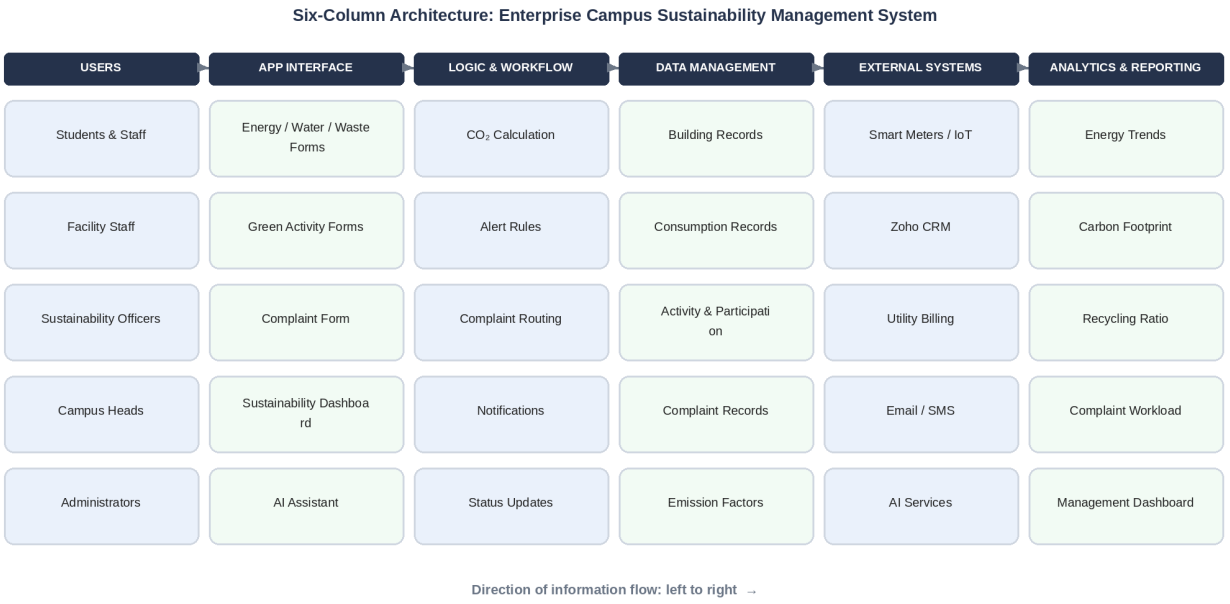


Figure 13.1: Six-Column Architecture of the Enterprise Campus Sustainability Management System

- **Users:** students and staff, facility staff, sustainability officers, campus heads and administrators, who interact with the system.
- **App Interface:** the energy, water and waste forms, green activity forms, the complaint form, the Sustainability Dashboard and the AI assistant.

- **Logic & Workflow:** CO_2 calculation, alert rules, complaint routing, notifications and status updates, which move a record through the process.
- **Data Management:** building records, consumption records, activity and participation records, complaint records and emission factors, in which the information is stored.
- **External Systems:** smart meters / IoT, Zoho CRM, utility billing, e-mail / SMS and AI services with which the application must exchange information.
- **Analytics & Reporting:** energy trends, carbon footprint, recycling ratio, complaint workload and a management dashboard for decision makers.

The groups interact in sequence. Users enter and view information through the application; the logic and workflow layer processes it; the data layer stores and supplies it; external systems provide or receive related information such as meter readings, customer relationships and utility bills; and the analytics layer reports on the results.

How a Low-Code Platform Can Support the System

A large part of the system is standard application functionality that low-code platforms are designed to handle:

Capability	Use in the Campus Sustainability System
Forms	Energy_Consumption, Water_Consumption, Waste_Management and Sustainability_Complaints capture readings, quantities, dates, building lookups and complaint details.
Reports	Reports and their display formats (list, Kanban, pivot, charts) show consumption by building, status and category.
Workflows	Calculate Estimated CO_2, block abnormal readings, set the energy status and push complaints to the CRM.
Business process automation	Rules and automatic actions, such as updating a status or computing a recycling percentage when a record is saved.
Data management	Stores building, consumption, activity and complaint records and performs standard create, read, update and delete (CRUD) operations.
User interfaces	Web and mobile screens for staff and students, with a dashboard of KPI tiles and charts.
Roles and sharing	Role hierarchy (Sustainability Officer → Facility Coordinator → Data Entry Staff) and a Read Only data-sharing rule control who sees which records.
Basic integrations	Connect to common services through built-in connectors, such as the Zoho CRM connection used in Experiment 7.

The Eight Core Limitations at Enterprise Scale

As the system grows into a complex enterprise platform, the following limitations become significant. Figure 13.2 summarises how these limitations arise: a basic low-code application grows into growing enterprise requirements and then enterprise-scale challenges, which lead to the eight key limitations.

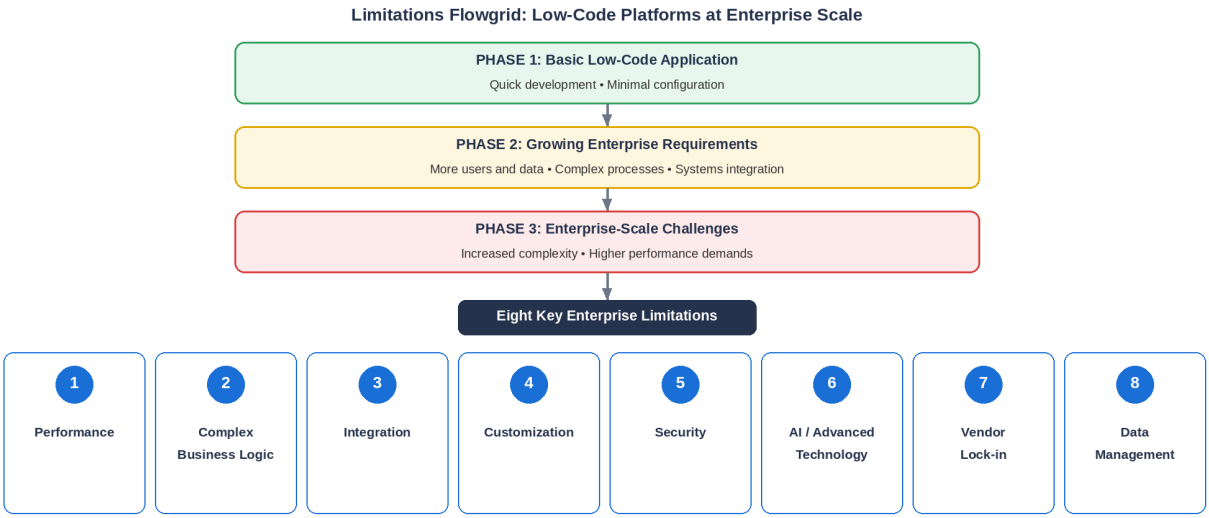


Figure 13.2: Limitations Flowgrid – Limitations of Low-Code Platforms at Enterprise Scale

1. Performance and Scalability

When meter readings, records and concurrent users grow very large, for example thousands of smart-meter readings arriving every few minutes from many campuses, performance may be affected. Complex queries and reports over large amounts of data respond slowly, and the platform may impose limits on how far an application can scale. Because the underlying infrastructure is managed by the platform, the options for tuning performance are limited.

2. Complex Business Logic

Highly specialised rules, such as emission factors that differ by region, energy source and time of day, multi-level approval of sustainability targets, or optimised allocation of maintenance teams under skill, location and workload constraints, are difficult to express with visual workflows and standard scripting. Such requirements may need custom development.

3. System Integration

Connecting the application to smart meters and building management systems, ERP and CRM systems, utility billing platforms, legacy campus systems and messaging services can be challenging. Built-in connectors may not cover every system or data format, and complex or real-time integrations may need custom interfaces.

4. Extreme UX Customization

Standard components provide common screens and layouts. Highly specialised interfaces, such as a live digital-twin view of the campus, a branded public sustainability portal, or a mobile app that scans meters, may exceed what the standard components can provide.

5. Security and Governance

Institutions require detailed access control, governance, compliance and strong security practices. Role- and permission-based controls, such as the role hierarchy and Read Only sharing rule used here, may need to be supplemented to meet fine-grained or organisation-specific requirements for personal data in complaints and for audit of environmental reporting, and governance of many applications must be managed carefully.

6. AI / ML and Advanced Technology

Specialised artificial-intelligence and machine-learning models, such as forecasting energy demand, detecting anomalies in meter data or estimating emissions from satellite data, may require external services or custom code, since a low-code platform may not provide such capabilities directly. In this project the assistant is rule-based (it answers from the stored data using Deluge), and the OpenAI connection was only tested; larger AI features would need external services.

7. Vendor / Platform Dependency

The application depends on the capabilities, restrictions, pricing and supported integrations of the selected platform. Features may be limited by the plan or edition (for example the plan restrictions seen in earlier experiments), and moving the application to another platform later can be difficult.

8. Heavy Data Transformations

Large datasets, complex relationships between records and advanced processing, such as bulk import of years of meter data, aggregation across campuses or complex transformations for regulatory sustainability reporting, may be difficult to handle within the data model and tools provided by the platform.

Relevance of each limitation to the case study

Limitation	Relevance to the Campus Sustainability System
Performance & Scalability	Growing numbers of buildings, meter readings and complaints, and reports over years of consumption history.
Complex Business Logic	Emission factors, targets and maintenance allocation rules that become more complex over time.
System Integration	Smart meters, CRM, utility billing and e-mail / SMS services.
UX Customization	Specialised dashboards for campus heads and a tailored experience for students.
Security & Governance	Different access needs for students, staff, facility teams and sustainability officers; protection of personal data in complaints.
AI / Advanced Technology	Possible demand forecasting and anomaly detection built on consumption data.
Vendor Dependency	Reliance on the platform for all application, workflow and reporting features, and on its plan limits.
Heavy Data Transformations	Bulk imports and large volumes of consumption history across campuses.

Low-Code vs Pro-Code Requirements

Neither approach is universally better; the comparison below indicates the typical role of each.

Area	Low-Code Approach	When Pro-Code May Be Required
User interface	Standard screens, forms and layouts from predefined components.	Highly specialised interfaces beyond standard components.
Business logic	Visual workflows, rules and simple scripting.	Complex rules and advanced processing that visual tools cannot express.
Data management	Built-in data model with standard CRUD operations.	Very large datasets, complex relationships and advanced data processing.
Integrations	Built-in connectors and standard interfaces.	Legacy, specialised or real-time integrations and systems without connectors.

Area	Low-Code Approach	When Pro-Code May Be Required
AI / ML	Basic features offered by the platform, where available.	Custom AI models, ML pipelines and specialised analytics.
Security	Role- and permission-based access control provided by the platform.	Fine-grained access control and compliance beyond platform features.
Scalability	Suitable where the platform handles the expected users and data.	High data volumes, large user numbers and demanding performance.

Resolution Topology: Three-Tier Hybrid Architecture

Organisations can combine low-code development with pro-code extensions. Low-code is used to develop standard application functionality quickly, while custom code, APIs, integrations and specialised processing are used for requirements that exceed the built-in capabilities of the platform. The structure consists of three layers: Low-Code Application → Pro-Code Extensions → Enterprise Systems. Figure 13.3 shows this architecture: users access the system at the top, and information flows in both directions between the layers.

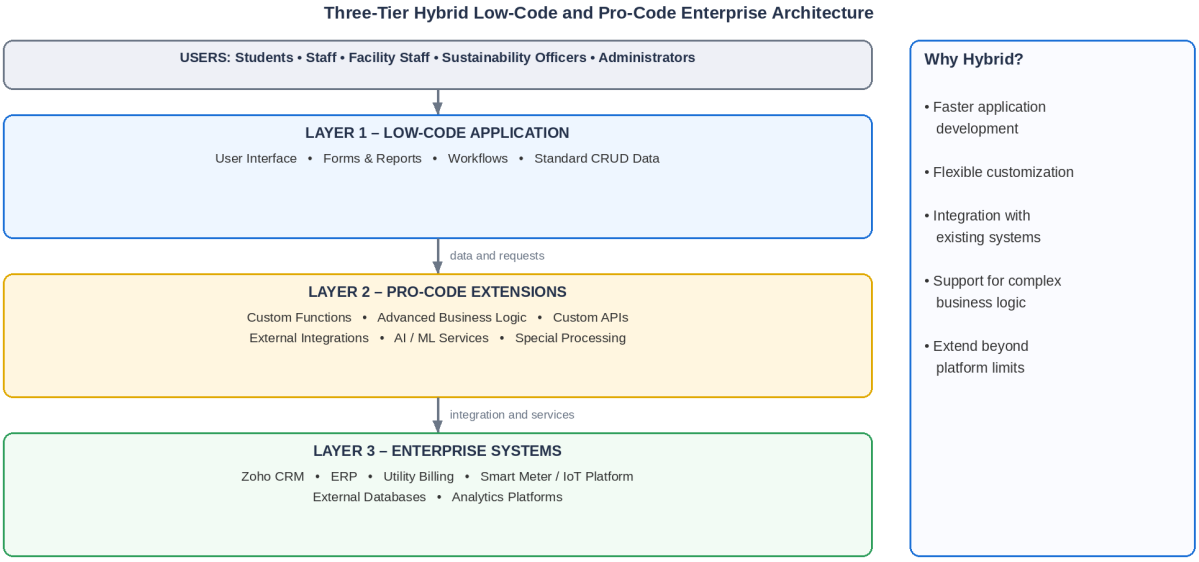


Figure 13.3: Three-Tier Hybrid Low-Code and Pro-Code Enterprise Architecture

Layer	Role in the Campus Sustainability System
Layer 1 – Low-Code Application	Rapid development of standard needs: user interface, forms and reports, workflows, role-based access and standard CRUD data management. This is the application built in Zoho Creator.
Layer 2 – Pro-Code Extensions	Custom functions, advanced business logic (for example region-specific emission factors), custom APIs, external integrations, AI / ML services (for example demand forecasting) and special processing such as bulk data transformations.
Layer 3 – Enterprise Systems	Core systems providing data, services and infrastructure: Zoho CRM, ERP, utility billing, smart-meter platforms, external databases and analytics platforms.

The “Why Hybrid?” panel of the figure lists the five reasons for a hybrid approach: faster application development, flexible customisation, integration with existing systems, support for complex business logic, and the ability to extend beyond platform limits.

When Pro-Code Extension May Become Necessary

- **Complex business logic:** when rules cannot be represented with standard workflows and rules.
- **Specialised integrations:** when the application must connect to legacy, ERP or other systems without suitable connectors.
- **Custom APIs:** when services or interfaces must be exposed or consumed in a way the platform does not support directly.
- **Advanced AI / ML:** when intelligent features need custom models or ML pipelines.
- **High-performance processing:** when large batches or real-time processing exceed what the platform handles efficiently.
- **Specialised user interfaces:** when the required experience goes beyond standard components.
- **Complex enterprise data processing:** when large datasets and complex relationships need advanced data handling.
- **Requirements beyond platform capabilities:** when needed functionality is simply not offered by the platform.

Case Study Analysis

For the Campus Sustainability System, a low-code platform can be used effectively to develop the standard parts rapidly: the energy, water and waste forms, reports on consumption, workflows for CO_2 calculation and alerts, complaint handling, role-based access and dashboards with KPI tiles and charts. These correspond to what low-code platforms are designed to provide, and are what the preceding experiments built.

As the organisation grows, requirements become more demanding. Large numbers of buildings and readings raise performance and data-management concerns; integration with smart meters, CRM and billing systems raises integration concerns; differing access needs and the handling of personal data raise security and governance concerns; and advanced features such as demand forecasting need external services. In such cases extensions, integrations or custom development are needed in addition to the low-code application. Low-code is therefore useful for rapidly developing standard business applications, whereas increasingly complex enterprise requirements call for a hybrid approach, and the choice should be based on the requirements of the application.

Result

The case study demonstrated the major limitations that arise when low-code platforms are applied to complex enterprise requirements: performance and scalability, complex business logic, system integration, extreme UX customisation, security and governance, AI / ML and advanced technology, vendor or platform dependency, and heavy data transformations. It also showed how a three-tier hybrid architecture of a low-code application, pro-code extensions and enterprise systems can address requirements beyond standard low-code capabilities.

Conclusion

Low-code platforms offer speed and simplicity for standard business applications, but their limitations become more significant as enterprise complexity increases. The trade-off is clear: low-code gives fast delivery, lower cost and wider participation in development, in exchange for ceilings in scale, logic, integration, customisation, governance and platform independence. For the Campus Sustainability

Management System, the standard functionality can be built quickly with low-code, while demanding requirements in logic, integration, security, scale and advanced technology need pro-code extensions connected to core enterprise systems. A hybrid architecture keeps the speed of low-code for the standard layer and adds the flexibility of custom code where it is needed, which is why it is regarded as the practical standard for scalable enterprise platforms. It is therefore important to understand the limitations of a platform before selecting a development approach, so that the chosen approach matches the requirements of the enterprise application.